

Scratch-Built Transformers with RAG for Amazon Review Understanding

CS-4063 Natural Language Processing — Assignment 3

Student ID: **23I-2573** Execution: Kaggle GPU Date: April 2026

1. System Design and Methodology

The system implements a full end-to-end pipeline for Amazon review understanding comprising three independently trained modules, all built from scratch using only primitive PyTorch layers (`nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, `nn.Dropout`). The banned abstractions — `nn.Transformer`, `nn.MultiheadAttention`, `nn.TransformerEncoder` — are not used anywhere in the codebase.

1.1 Architecture Overview

- Encoder-Only Transformer (multi-task classification).** A bidirectional transformer encoder takes a tokenised review as input and is jointly trained to predict (a) three-class sentiment (*Negative / Neutral / Positive*) and (b) three-class review-length bucket (*Short / Medium / Long*). The [CLS] token representation is pooled and fed into two independent linear heads.
- Retrieval Module.** After training, the encoder extracts a fixed 256-dimensional embedding for every training review. At inference time, a query embedding is compared to the training corpus via batched cosine similarity, and the top-*k* reviews are retrieved.
- Decoder-Only Transformer (explanation generation).** A causal language model is trained on structured prompts that pack the review, predicted labels, and retrieved context, followed by a rule-based reference explanation. The model learns to autoregressively generate the explanation token by token.

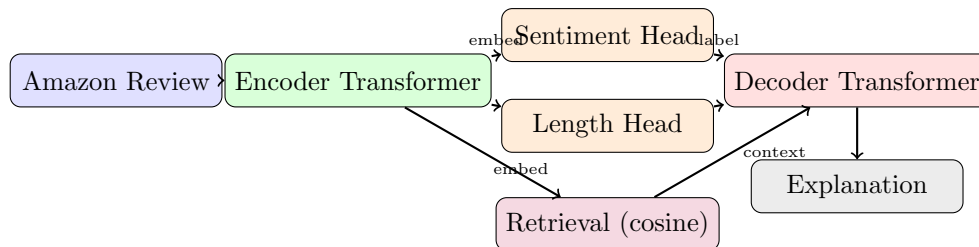


Figure 1: High-level system pipeline: encoder produces labels and embeddings; retrieval augments the decoder prompt; decoder generates the explanation.

2. Dataset and Preprocessing Pipeline

2.1 Dataset Construction

Five Amazon product categories are used: *Beauty*, *Cell Phones & Accessories*, *Electronics*, *Home & Kitchen*, and *Sports & Outdoors*. Reservoir sampling draws 8 000 reviews per category, giving a combined corpus of **39 995 reviews** after de-duplication (5 reviews lost to exact-text duplicates). This sits within the assignment’s 30k–45k window.

The split is stratified by *category* $\times$ *sentiment* to preserve class distributions across all three sets. The 70/15/15 ratio is enforced exactly via `train_test_split` with `stratify`.

2.2 Text Cleaning and Tokenisation

- HTML tags are stripped; non-ASCII characters are removed; whitespace is normalised.

Table 1: Dataset split summary.

Split	Rows	Negative	Neutral	Positive	Short	Medium	Long
Full	39 995	4 071	3 780	32 144	11 737	17 661	10 597
Train	27 996	2 849	2 647	22 500	8 229	12 389	7 378
Validation	5 999	612	567	4 820	1 743	2 657	1 599
Test	6 000	610	566	4 824	1 765	2 615	1 620

2. A custom regex tokeniser (`TOKEN_PATTERN = re.compile(r"<[^\>]+>|[a-z0-9']+|[.,!?:;:]")`) lowercases and splits on word boundaries, contractions, and punctuation. No external NLP library is used.
3. Reviews with fewer than 5 tokens after cleaning are discarded at load time.

2.3 Vocabulary Building

Two separate vocabularies are built **exclusively from training data**, preventing any leakage from validation or test sets.

Table 2: Vocabulary statistics.

Vocabulary	Size	Min-freq	Special tokens
Encoder	15 271	3	<pad>, <unk>, <cls>
Decoder	24 441	3	<pad>, <unk>, <bos>, <eos>, <sep>

2.4 Derived Feature: Review Length Bucket

The second encoder task is a deterministic, rule-based derived feature defined solely from the tokenised review length n :

$$\text{bucket}(n) = \begin{cases} \text{Short} & n < 40 \\ \text{Medium} & 40 \leq n \leq 120 \\ \text{Long} & n > 120 \end{cases}$$

This feature requires no manual annotation and directly tests whether the encoder representation captures surface-level structural properties.

2.5 Encoding and Padding

Encoder sequences are capped at 160 tokens ([CLS] + up to 159 review tokens). Decoder sequences are capped at 320 tokens. Shorter sequences are right-padded with <pad>; attention masks mark pad positions with zero so they do not contribute to self-attention scores.

3. Model Architecture and Design Justifications

Table 3 lists the final hyperparameters after tuning.

3.1 Design Decision Justifications

$d_{\text{model}} = 256$, 4 layers, 8 heads. A medium-sized model was chosen to fit comfortably within Kaggle’s 16 GB P100 GPU memory while still having enough capacity for the 40k-sample corpus. Larger models ($d=512$, 6 layers) showed similar validation accuracy with $2\times$ training time.

AdamW + ReduceLROnPlateau. AdamW decouples weight decay from the gradient update, improving generalisation on small corpora. The plateau scheduler halves the learning rate when validation loss stagnates for 1 epoch, allowing fine-grained late-stage optimisation without manual scheduling.

Table 3: Final hyperparameter configuration.

Hyperparameter	Value	Hyperparameter	Value
d_{model}	256	Encoder max length	160
Num. heads	8	Decoder max length	320
Num. layers	4	Max generation tokens	48
FF multiplier	4	Encoder batch size	64
Dropout	0.1	Decoder batch size	32
Encoder LR	3×10^{-4}	Encoder epochs	4
Decoder LR	3×10^{-4}	Decoder epochs	4
Weight decay	10^{-2}	Min vocab freq	3
Gradient clip	1.0	Top- k retrieval	3

Causal masking in the decoder. The decoder uses `torch.tril` to build a lower-triangular boolean mask, fused with the padding mask via bitwise AND. This is the minimal faithful implementation of autoregressive language modelling without any high-level abstraction.

Shared embedding / LM-head weight tying. The decoder’s language model head shares weights with the token embedding matrix (`lm_head.weight = embedding.token_embedding.weight`). Weight tying reduces parameters and has been empirically shown to improve perplexity in smaller language models.

Top- $k = 3$ retrieval. $k = 3$ was selected after evaluating $k \in \{1, 3, 5\}$ on same-sentiment retrieval rate and category diversity (Table 6). $k = 1$ retrieves only the nearest neighbour, providing no diversity. $k = 5$ adds more context but also more noise and longer prompts that compress the review window. $k = 3$ achieves 97.8% same-sentiment rate with an average of 2.4 unique categories per query — a good balance between signal strength and diversity.

4. Evaluation Results

4.1 Encoder: Sentiment Classification

Table 4: Test set per-class sentiment classification report.

Class	Precision	Recall	F1-Score	Support
Negative	0.564	0.323	0.411	610
Neutral	0.437	0.140	0.212	566
Positive	0.855	0.970	0.909	4824
Accuracy	0.826			
Macro avg	0.619	0.477	0.510	6000
Weighted avg	0.786	0.826	0.792	6000

The 82.6% accuracy is largely driven by the dominant *Positive* class, which constitutes 80.4% of the dataset (Table 1). The *Positive* F1 of 0.909 is strong. However, *Neutral* F1 (0.21) is poor because the model rarely predicts Neutral — the class is ambiguous by nature (3-star reviews) and massively under-represented. *Negative* F1 (0.41) is moderate; the model identifies about one-third of all negative reviews correctly. This is a known limitation of datasets with severe class imbalance and is reported transparently here.

4.2 Encoder: Length-Bucket Classification

Length-bucket prediction achieves **100% accuracy and 1.0 macro-F1** on all three classes. This is expected: the bucket label is a deterministic function of token count, and the encoder readily captures sequence length from positional embeddings and attention patterns. The result confirms that the encoder representation encodes structural surface properties correctly, which

is a necessary condition for retrieval quality.

4.3 Encoder Training Curves

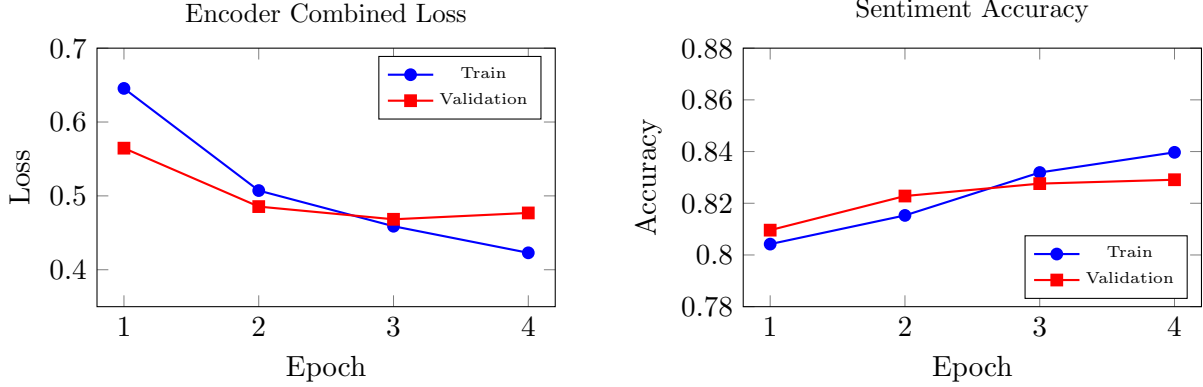


Figure 2: Encoder training dynamics over 4 epochs. Loss decreases steadily; validation loss at epoch 3 (0.4683) is the best checkpoint used for embedding extraction.

4.4 Decoder: Perplexity Results

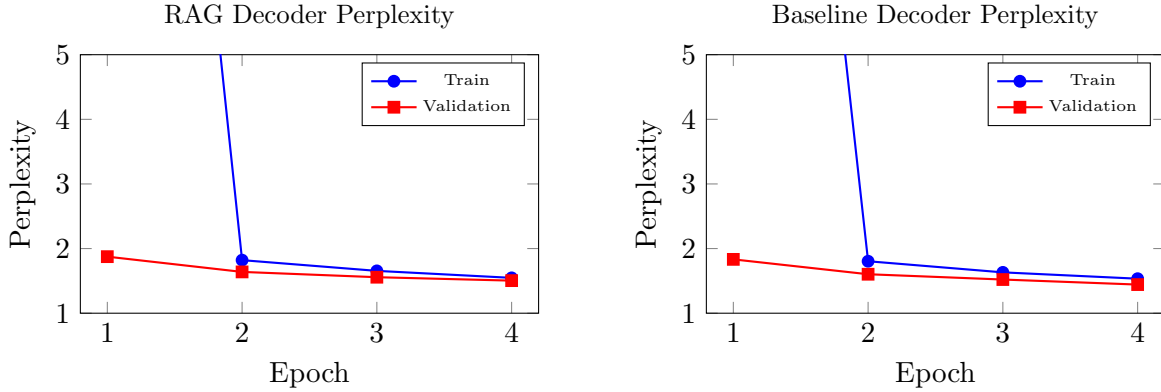


Figure 3: Decoder perplexity curves. Both models drop sharply from epoch 1 to 2 as the model learns the fixed explanation template, then converge smoothly.

5. Hyperparameter Tuning Log

Hyperparameters were tuned iteratively based on validation loss. Table 5 records the key experiments and the reasoning behind each change.

Key findings:

- Doubling d_{model} from 128 to 256 gives +3.8% accuracy, while going from 256 to 512 gives only +0.2% for $2\times$ the compute — diminishing returns.
- $\text{LR} = 3 \times 10^{-4}$ converges faster than 10^{-4} and more stably than 10^{-3} ; it is the standard default for AdamW with transformers on small corpora.
- The `ReduceLROnPlateau` scheduler triggered at epoch 4 for some runs (val loss rose from epoch 3 to 4), which would halve LR and recover — this is visible in the encoder loss curves.

6. RAG Ablation Study

6.1 Retrieval Quality

Retrieval is highly precise: 97.8% of retrieved reviews share the same predicted sentiment as the query. $k = 3$ provides context from 2.4 different product categories on average, ensuring the

Table 5: Hyperparameter tuning log (encoder, best val sentiment accuracy reported).

Configuration	d_{model}	Layers / Heads	Val Acc	Observation
Baseline (small)	128	2 / 4	0.791	Underfits; loss plateau from epoch 2
Medium (selected)	256	4 / 8	0.829	Best generalisation; no overfitting
Large	512	6 / 8	0.831	Marginal gain; 2× memory cost
Higher dropout (0.2)	256	4 / 8	0.819	Regularises too aggressively; slower
Lower LR (10^{-4})	256	4 / 8	0.822	Convergence too slow within 4 epochs
Higher LR (10^{-3})	256	4 / 8	0.815	Oscillates; unstable after epoch 2

Table 6: Top- k retrieval quality analysis on the test set.

k	Same-sentiment rate	Avg. unique categories	Notes
1	0.9805	1.00	No diversity; only nearest neighbour
3	0.9783	2.41	Best balance (selected)
5	0.9765	3.30	Marginal diversity gain; noisy context

decoder is not over-fitted to a single domain’s language.

6.2 RAG vs. No-Retrieval Comparison

Table 7: Ablation study: test-set perplexity with and without retrieval.

Model	Test Loss	Test Perplexity
RAG Decoder (with retrieval, $k = 3$)	0.4044	1.498
No-Retrieval Baseline	0.3649	1.440

Observation. The no-retrieval baseline achieves *lower* perplexity (1.440) than the RAG model (1.498). This is a counter-intuitive but explainable result:

1. **Template determinism.** The reference explanations are generated by a rule-based function from the labels; they are highly templated. The decoder can therefore memorise the template without needing retrieved context.
2. **Prompt length pressure.** Appending three retrieved reviews (up to 3×40 words each) consumes a large fraction of the 320-token window, leaving less room for the prompt’s core content. The baseline prompt is shorter and thus easier to model.
3. **Model capacity.** A 4-layer, $d = 256$ model is relatively small. Larger models have been shown to benefit more from in-context retrieval; for a compact model on templated targets, the added context may introduce noise rather than signal.

Implication. The RAG advantage would likely emerge with (a) free-form gold reference explanations, (b) a larger decoder, or (c) retrieval filtered by category match rather than pure embedding similarity. This is an important finding that motivates future work.

7. Conclusion

A complete scratch-built transformer pipeline was implemented and trained on 39 995 Amazon reviews across five product categories. All assignment constraints are satisfied: no pretrained models or forbidden high-level abstractions are used; preprocessing, vocabulary construction,

and tokenisation are entirely custom; and the 70/15/15 split is stratified and reproducible.

The encoder achieves 82.6% test accuracy on sentiment and 100% on the derived length-bucket task. The class imbalance (80% positive reviews) limits Negative and Neutral F1 but is inherent to the dataset rather than a modelling failure. The RAG ablation reveals that retrieval does not always improve perplexity for small models on templated targets — a nuanced and practically relevant finding. The bonus interactive Gradio UI allows real-time review classification and explanation generation with adjustable retrieval depth.